# CSC-239

# *Java Programming*: Chapter 4: Mathematical Functions, Characters, and Strings

September 15, 2025

**NOTICE to Students**: Portions of these lecture slides are
Copyright 2021 Pearson Education, Inc.
We have a license to use this material ***only*** in the context of this course.
There is NO PERMISSION to share these lecture slides with anyone not taking the course.
There is NO PERMISSION to post these lecture slides on the Internet.

# Reminder

- Quiz 1 will begin on Monday, September 15.
  - Online (Moodle) Quiz.
  - The Quiz will open Monday evening, at 9:00 p.m.
  - The Quiz will remain open for approximately one week.
  - Each student will have two hours to complete the Quiz, from the time that they begin.
- The material covered on Quiz 1 will be
  - The content of the course Syllabus
  - The lectures of September 3 and 8.
  - The quiz will include True / False, and Multiple-Choice questions.

# Reminder: Submitting Files to *Moodle*

- When you submit your source code (*.java) files to *Moodle*, you MUST follow the **Submit Assignment to *Moodle*** procedure *completely*: be sure that the final Submission Status is:

  **"Submitted for grading".**




- This procedure is described in detail in the **Submit_Assignment_to_Moodle.pdf** document, which is available on *Moodle*, under the "**Handouts**" topic.

# Online Information

- Java Tutorials:

  https://docs.oracle.com/javase/tutorial/

- Java API:

  https://docs.oracle.com/en/java/javase/21/docs/api/index.html

- The official "*Grader Than*" documentation:

  https://docs.graderthan.com/student/

- *Grader Than* Support E-mail:

  support@graderthan.com

# The Math Class

- Class constants:
  - `PI`
  - `E`
- Class methods:
  - Trigonometric Methods
  - Exponent Methods
  - Rounding Methods
  - min, max, abs, and random Methods

# Trigonometric Methods

- `sin(double a)`
- `cos(double a)`
- `tan(double a)`
- `acos(double a)`
- `asin(double a)`
- `atan(double a)`

# Degrees and Radians

- Most people like to think of angles as being measured in degrees
- The Trigonometric methods expect angles to be represented in radians
- Conversion methods:

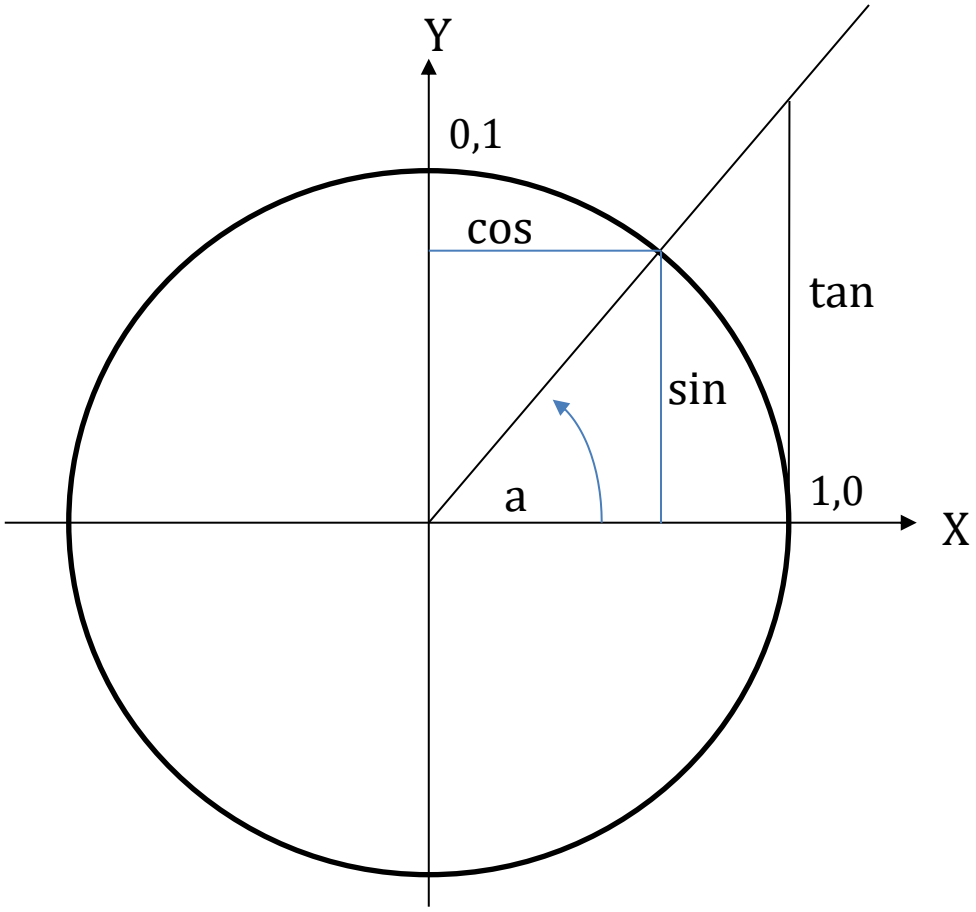
```
toRadians(double degrees)
toDegrees(double radians)
```

# “Unit Circle” Trigonometry

$$x = \cos(a)$$

$$y = \sin(a)$$

$$\tan(a) = y/x$$



# Different Perspectives

- If you usually think of angles in terms of a "compass heading", or the minutes on a clock face, then you are probably accustomed to thinking of the zero angle as pointing straight up.
- To use the trigonometry methods, you need to adjust your frame of reference to the convention used by the Math class:

  An angle with value of zero points to the right, along the X-axis.

# TrigDemo.java

- A simple program that displays the values of the basic trigonometry methods in terms of "compass heading" angles, expressed as degrees and radians.

# Exponent Methods

**`exp(double a)`**

Returns **e** raised to the power of a.

**`log(double a)`**

Returns the natural logarithm of **a**.

**`log10(double a)`**

Returns the 10-based logarithm of **a**.

**`pow(double a, double b)`**

Returns **a** raised to the power of **b**.

**`sqrt(double a)`**

Returns the square root of **a**.

# Rounding Methods

**`double ceil(double x)`**

x rounded up to its nearest integer. This integer is returned as a double value.

**`double floor(double x)`**

x is rounded down to its nearest integer. This integer is returned as a double value.

**`double rint(double x)`**

x is rounded to its nearest integer. If x is equally close to two integers, the even one is returned as a double.

**`int round(float x)`**

Return (int)Math.floor(x+0.5).

**`long round(double x)`**

Return (long)Math.floor(x+0.5).

# The random Method

- Generates a random double value greater than or equal to 0.0 and less than 1.0:

  (0 <= Math.random() < 1.0)

- Multiply and/or add values to the random() result to achieve the desired range of values:

  a + Math.random() * b

  (a = offset , b = range)

# Comparing and Testing Characters

In this example, assume:

1. The **ch** variable is of type **char**:

```
char ch;
```

and

2. The **ch** variable has been set to some valid value.

```
if (ch >= 'A' && ch <= 'Z')
  System.out.println(ch + " is uppercase");
else if (ch >= 'a' && ch <= 'z')
  System.out.println(ch + " is lowercase");
else if (ch >= '0' && ch <= '9')
  System.out.println(ch + " is numeric");
```

# Methods in the Character Class

| Method | Description |
|---|---|
| **isDigit(ch)** | Returns **true** if the specified character is a digit. |
| **isLetter(ch)** | Returns **true** if the specified character is a letter. |
| **isLetterOrDigit(ch)** | Returns **true** if the specified character is a letter or a digit. |
| **isLowerCase(ch)** | Returns **true** if the specified character is a lowercase letter. |
| **isUpperCase(ch)** | Returns **true** if the specified character is an uppercase letter. |
| **toLowerCase(ch)** | Returns the lowercase of the specified character. |
| **toUpperCase(ch)** | Returns the uppercase of the specified character. |

# The String Type

- For representing multiple characters.
- **String** is a predefined library class (like **System** and **Scanner**)
- The methods listed on the next slide are a subset of those available.
- See also the online documentation:

https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/String.html

# Simple Methods for **String** Objects

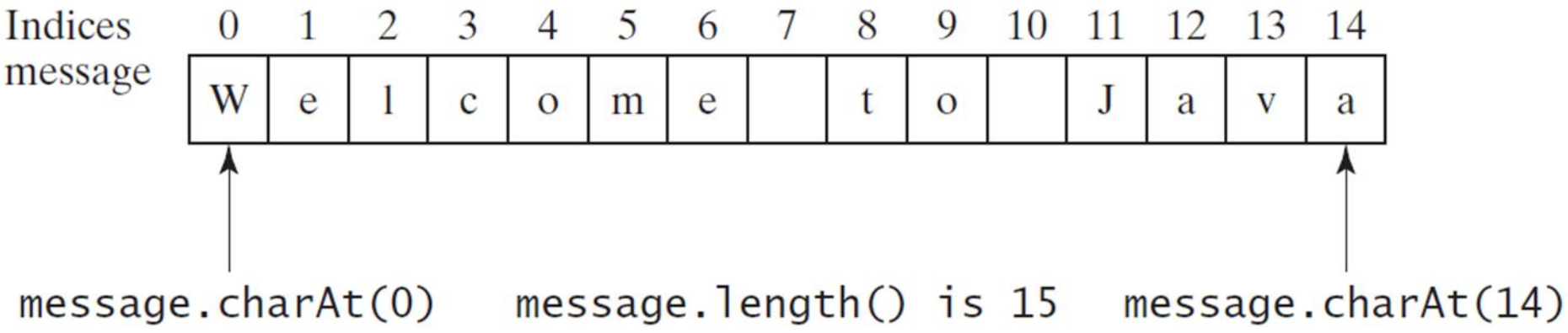
| Method | Description |
|---|---|
| **length()** | Returns the number of characters in this string. |
| **charAt(index)** | Returns the character at the specified index from this string. |
| **concat(s1)** | Returns a new string that concatenates this string with string s1 |
| **toUpperCase()** | Returns a new string with all letters in uppercase. |
| **toLowerCase()** | Returns a new string with all letters in lowercase. |
| **trim()** | Returns a new string with whitespace characters trimmed on both sides. |

# Getting String Length

```
String message = "Welcome to Java";
System.out.println("The length of "
                   + message + " is "
                   + message.length());
```

# Getting Characters from a String

```
String message = "Welcome to Java";
System.out.println("The first character is "
   + message.charAt(0));
```



# Converting Strings

```
"Welcome".toLowerCase()
```

returns a new string: **welcome**

```
"Welcome".toUpperCase()
```

returns a new string: **WELCOME**

```
"   Welcome   ".trim()
```

returns a new string: **Welcome**

# Reading a String from the Console

```
Scanner input = new Scanner(System.in);
System.out.print("Enter 3 words:");
String s1 = input.next();
String s2 = input.next();
String s3 = input.next();
System.out.println("s1 is " + s1);
System.out.println("s2 is " + s2);
System.out.println("s3 is " + s3);
```

# Read a line, isolate first character

```
Scanner input = new Scanner(System.in);
System.out.print("Enter a character: ");
String s = input.nextLine();
char ch = s.charAt(0);
System.out.println("character is " + ch);
```

# Comparing Strings

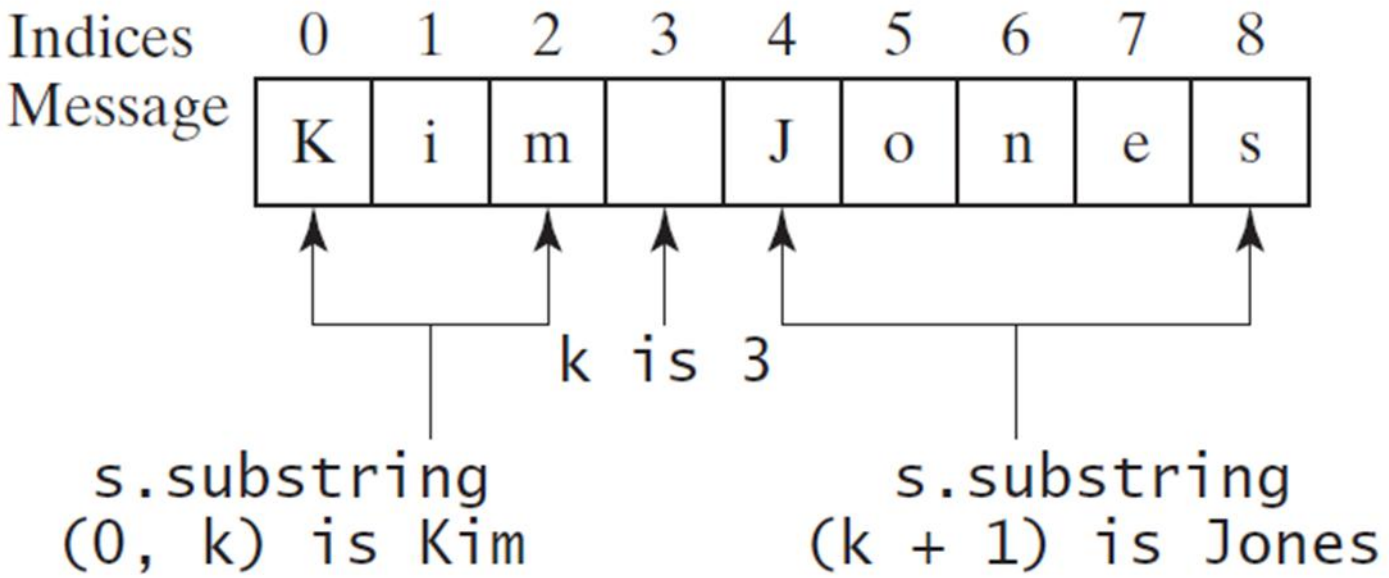
| Method | Description |
|---|---|
| **equals(s1)** | Returns **true** if this string is equal to s1. (case sensitive) |
| **equalsIgnoreCase(s1)** | Returns **true** if this string is equal to s1. (case insensitive) |
| **compareTo(s1)** | Returns an integer greater than 0, equal to 0, or less than 0 to indicate whether this string is greater than, equal to, or less than s1. |
| **compareToIgnoreCase(s1)** | Same as **compareTo**, except that the comparison is case insensitive. |
| **startsWith(prefix)** | Returns **true** if this string starts with the specified prefix. |
| **endsWith(suffix)** | Returns **true** if this string ends with the specified suffix. |

# Finding a Character in a String

| Method | Description |
|---|---|
| **indexOf(ch)** | Returns the index of the first occurance of **ch** in the string. Returns -1 if not matched. |
| **indexOf(ch, fromIndex)** | Same as **indexOf(ch)**, but starts searching at position **fromIndex**. |
| **indexOf(s, fromIndex)** | Same as **indexOf(ch, fromIndex)**, but seaches for the first occurrence of string **s**. |
| **lastIndex(ch)** | Returns the index of the last occurance of **ch** in the string. Returns -1 if not matched. |
| **lastIndex(s)** | Returns the index of the last occurance of string **s** in the string. Returns -1 if not matched. |
| **lastIndex(s, fromIndex)** | Same as **lastIndex(s)**, but seaches for the last occurrence of string **s**, before position **fromIndex**. |

# Extracting a substring

```
int k = s.indexOf(' ');
String firstName = s.substring(0, k);
String lastName = s.substring(k + 1);
```



# Converting a String to a Number

```
int intValue =
  Integer.parseInt(intString);
```

```
double doubleValue =
  Double.parseDouble(doubleString);
```